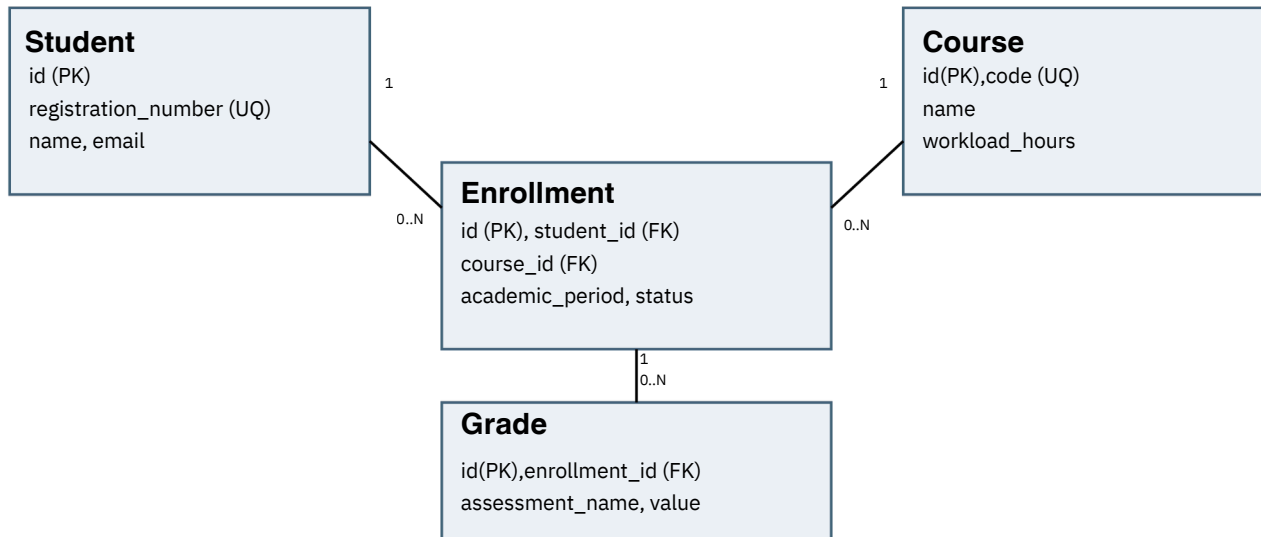


Dados e Relacionamentos

Modelagem acadêmica

DER: entidades e cardinalidades



Atributos e Regras

Dicionário e invariantes

Entidade	Atributos SQL (todos NOT NULL)
Student	id UUID; registration_number TEXT; name TEXT; email TEXT
Course	id UUID; code TEXT; name TEXT; workload_hours INT
Enrollment	id UUID; student_id UUID; course_id UUID; academic_period TEXT; status enrollment_statuses
Grade	id UUID; enrollment_id UUID; assessment_name TEXT; value NUMERIC

Garantias no Banco

Os quatro id são PK com `gen_random_uuid()`. Matrícula institucional e código da disciplina são únicos. A combinação (student_id, course_id, academic_period) é única mesmo se cancelada. Reativação reutiliza o vínculo; outro período permite nova matrícula.

Textos obrigatórios rejeitam vazio e apenas espaços comuns via `trim`. Carga horária é inteira e maior que zero. Nota é obrigatória e fica no intervalo inclusivo de 0 a 10, sem escala decimal fixa. O estado padrão é `Active`; o enum restringe as alternativas.

```
CHECK ((value>=0)AND (value <= 10))
```

Três FKs usam `ON DELETE RESTRICT`: não se apagam registros com dependentes. Atualizações de chaves usam `NO ACTION` por omissão. Índices de `course_id` e `enrollment_id` apoiam consultas; o índice `UNIQUE` composto já começa por `student_id`.

Limites Deliberados

E-mail não é único nem validado por formato. Não há normalização de caixa, tabs ou Unicode. Período é texto sem formato imposto. Nomes de avaliações podem repetir; notas em vínculos cancelados são permitidas. Não há controle de transições, turmas, pesos, média, frequência ou aprovação. Essas regras dependem de requisitos posteriores, não foram inventadas na correção.

Resumo estrutural: 17 colunas obrigatórias, quatro PKs, três FKs, três `UNIQUE` e nove `CHECKs`. Tipos Haskell comuns não impedem, sozinhos, construir nota 11 em memória; a constraint é a garantia na persistência.

Records e tipos algébricos

Modelo de tipos e alinhamento didático

Records são tipos produto: agrupam atributos simultâneos. EnrollmentStatus é um tipo soma: escolhe uma entre três alternativas. O esquema abaixo é didático; o IHP gera os tipos reais a partir de Application/Schema.sql.

```
data EnrollmentStatus = Active | Cancelled | Completed
```

```
    deriving (Eq, Show)
```

```
data Student = Student
  { id :: Id Student, registrationNumber :: Text
  , name :: Text, email :: Text }
```

```
data Course = Course
  { id :: Id Course, code :: Text
  , name :: Text, workloadHours :: Int }
```

```
data Enrollment = Enrollment
  { id :: Id Enrollment, studentId :: Id Student
  , courseId :: Id Course, academicPeriod :: Text
  , status :: EnrollmentStatus }
```

```
data Grade = Grade
  { id :: Id Grade, enrollmentId :: Id Enrollment
  , assessmentName :: Text, value :: Scientific }
```

TEXT corresponde a Text; INT a Int; NUMERIC a Scientific. FKs usam Id do modelo referenciado. NOT NULL dispensa Maybe nos campos de domínio. snake_case do SQL corresponde a camelCase nos campos Haskell.

Não adicionar estas declarações como módulos paralelos. O trecho omite imports, extensões para campos repetidos, aliases e metadados. Generated.Types expõe os tipos reais, com estruturas internas como Grade'. relationSupport = false foi mantido: preserva FKs e IDs tipados, desativando Include/fetchRelated.